Test 3: Kombineret RAG

Vi undersøger, om Claude kombinerer relationer og undervisningstekster.

Spørgsmål: I already know Java and SQL. I want to learn Spring Boot. What other concepts should I study first, and can you explain how they relate to Spring Boot using the course material?

03-Graph retrieval
Input

```
{
  "conceptId": "spring-boot"
}
```

Output

```
{
  "executionId": "38",
  "status": "success",
  "data": {
    "Format Graph Results": [
      {
        "tool": "graph_search",
        "found": false,
        "results": [],
        "message": "Concept not found in the knowledge graph."
      }
    ]
  }
}
```


02-Vector retrieval
Input

```
{
  "question": "What do I need to know before learning Spring Boot? Prerequisites and related concepts."
}
```

Output

```
{
  "executionId": "39",
  "status": "success",
  "data": {
    "Format Vector Results": [
      {
        "tool": "vector_search",
        "count": 3,
        "results": [
```

```
        {
          "chunkId": "spring-01",
          "content": "Spring Boot is a Java framework for building web applications and REST APIs. It provides dependency injection, automatic configuration and integration with databases.",
          "documentId": "spring-guide",
          "title": "Spring Boot Fundamentals",
          "source": "project-created-seed-data",
          "similarity": 0.7823287,
          "topics": [
            "Java",
            "REST API",
            "JDBC",
            "Spring Boot"
          ]
        },
        {
          "chunkId": "oop-01",
          "content": "Object-oriented programming organizes software using classes and objects. Important principles include encapsulation, inheritance, polymorphism and abstraction.",
          "documentId": "oop-guide",
          "title": "Object-Oriented Programming",
          "source": "project-created-seed-data",
          "similarity": 0.6425391,
          "topics": [
            "Java",
            "Object-Oriented Programming"
          ]
        },
        {
          "chunkId": "mvc-01",
          "content": "Model View Controller separates applications into three responsibilities. The Model manages data and business logic, the View handles presentation and the Controller processes requests.",
          "documentId": "mvc-guide",
          "title": "MVC Architecture",
          "source": "project-created-seed-data",
          "similarity": 0.64086187,
          "topics": [
            "MVC Architecture",
            "Spring Boot"
          ]
        }
      ]
    }
  ]
 }
}
```

03-Graph retrieval

Input


```
{
  "conceptId": "spring"
}
```


Output


```
{
  "executionId": "40",
  "status": "success",
  "data": {
    "Format Graph Results": [
      {
        "tool": "graph_search",
        "found": true,
        "results": [
          {
            "conceptId": "spring",
            "concept": "Spring Boot",
            "prerequisites": [
              "Java",
              "Object-Oriented Programming",
              "REST API",
              "JDBC",
              "MVC Architecture"
            ],
            "nextTopics": [],
            "source": "project-curated-learning-path"
          }
        ],
        "message": "Graph relationships retrieved successfully."
      }
    ]
  }
}
```


03-Graph retrieval

Input


```
{
  "conceptId": "java"
}
```


Output


```
{
  "executionId": "41",
  "status": "success",
  "data": {
    "Format Graph Results": [
      {
        "tool": "graph_search",
```

```
      "found": true,
      "results": [
        {
          "conceptId": "java",
          "concept": "Java",
          "prerequisites": [],
          "nextTopics": [
            "Object-Oriented Programming",
            "Spring Boot"
          ],
          "source": "project-curated-learning-path"
        }
      ],
      "message": "Graph relationships retrieved successfully."
    }
  ]
 }
}
```


02-Vector retrieval

Input


```
{
  "question": "What is REST API and how does it work?"
}
```


Output


```
{
  "executionId": "42",
  "status": "success",
  "data": {
    "Format Vector Results": [
      {
        "tool": "vector_search",
        "count": 3,
        "results": [
          {
            "chunkId": "rest-01",
            "content": "A REST API exposes resources through HTTP endpoints. Clients use HTTP methods to retrieve, create, update and delete resources. REST commonly uses JSON for data exchange.",
            "documentId": "rest-guide",
            "title": "REST API Fundamentals",
            "source": "project-created-seed-data",
            "similarity": 0.82385063,
            "topics": [
              "HTTP",
              "REST API",
              "Spring Boot"
            ]
```

```
        },
        {
          "chunkId": "http-01",
          "content": "HTTP is a protocol used for communication between clients and servers. Requests use methods such as GET, POST, PUT and DELETE. Responses include status codes such as 200 and 404.",
          "documentId": "http-guide",
          "title": "Understanding HTTP",
          "source": "project-created-seed-data",
          "similarity": 0.7021986,
          "topics": [
            "HTTP",
            "REST API"
          ]
        },
        {
          "chunkId": "spring-01",
          "content": "Spring Boot is a Java framework for building web applications and REST APIs. It provides dependency injection, automatic configuration and integration with databases.",
          "documentId": "spring-guide",
          "title": "Spring Boot Fundamentals",
          "source": "project-created-seed-data",
          "similarity": 0.67950416,
          "topics": [
            "Java",
            "REST API",
            "JDBC",
            "Spring Boot"
          ]
        }
      ]
    }
  ]
 }
}
```


02-Vector retrieval

Input


```
{
  "question": "What is JDBC and how does it connect Java to databases?"
}
```


Output


```
{
  "executionId": "43",
  "status": "success",
  "data": {
    "Format Vector Results": [
      {
```

```
      "tool": "vector_search",
      "count": 3,
      "results": [
        {
          "chunkId": "jdbc-01",
          "content": "JDBC allows Java applications to communicate with relational databases. Developers can execute SQL statements using JDBC connections and prepared statements.",
          "documentId": "jdbc-guide",
          "title": "Java Database Connectivity",
          "source": "project-created-seed-data",
          "similarity": 0.8691686,
          "topics": [
            "Java",
            "SQL",
            "JDBC"
          ]
        },
        {
          "chunkId": "sql-01",
          "content": "SQL is used to manage relational databases. SELECT retrieves data, INSERT adds records, UPDATE modifies records and DELETE removes records. JOIN combines related tables.",
          "documentId": "sql-guide",
          "title": "Introduction to SQL",
          "source": "project-created-seed-data",
          "similarity": 0.714591,
          "topics": [
            "SQL"
          ]
        },
        {
          "chunkId": "java-01",
          "content": "Java is an object-oriented programming language. It uses classes, objects, methods and variables. Java is commonly used to build backend applications and enterprise software.",
          "documentId": "java-guide",
          "title": "Introduction to Java",
          "source": "project-created-seed-data",
          "similarity": 0.7006716,
          "topics": [
            "Java",
            "Object-Oriented Programming"
          ]
        }
      ]
    }
  ]
}
```

}

02-Vector retrieval
Input

{
  "question": "What is MVC architecture and how does it relate to web applications?"
}

Output

{
  "executionId": "44",
  "status": "error",
  "error": "The service is receiving too many requests from you"
}

02-Vector retrieval
Input

{
  "question": "What is MVC architecture Model View Controller?"
}

Output

{
  "executionId": "45",
  "status": "error",
  "error": "The service is receiving too many requests from you"
}

02-Vector retrieval
Input

{
  "question": "Object-oriented programming principles encapsulation inheritance polymorphism"
}

Output

{
  "executionId": "46",
  "status": "error",
  "error": "The service is receiving too many requests from you"
}

Great news — I have everything I need from the knowledge graph and the course documents to give you a thorough answer! Let me put it all together for you.

🎯 Your Spring Boot Learning Path

Since you already know **Java** and **SQL**, you have a great head start! According to the **project-curated learning path**, Spring Boot has **5 prerequisites**. Here's what you still need to study, and how each one connects to Spring Boot:

✅ Already Known

Concept	Why It Matters for Spring Boot
Java	Spring Boot is a Java framework — your Java skills are the foundation for everything
SQL	Spring Boot apps query databases using SQL under the hood

---

📚 Concepts to Study Next

1. 🧩 Object-Oriented Programming (OOP)

> *"Object-oriented programming organizes software using classes and objects. Important principles include encapsulation, inheritance, polymorphism and abstraction."* — **OOP Guide (oop-01)**

Why it matters for Spring Boot: Spring Boot is deeply OOP-driven. Its core feature — **Dependency Injection** — relies on classes and interfaces. You'll define service classes, repository classes, and controller classes that all work together through OOP principles. Without a solid OOP foundation, Spring Boot's design patterns will feel confusing.

---

2. 🌐 REST API

> *"A REST API exposes resources through HTTP endpoints. Clients use HTTP methods to retrieve, create, update and delete resources. REST commonly uses JSON for data exchange."* — **REST API Fundamentals (rest-01)**

> *"HTTP is a protocol used for communication between clients and servers. Requests use methods such as GET, POST, PUT and DELETE. Responses include status codes such as 200 and 404."* — **Understanding HTTP (http-01)**

Why it matters for Spring Boot: One of Spring Boot's primary uses is **building REST APIs**. You'll use annotations like `@GetMapping`, `@PostMapping`, and `@RestController` constantly. Understanding HTTP methods (GET, POST, PUT, DELETE) and JSON is essential to make sense of what Spring Boot is doing.

---

3. 🗃️ JDBC (Java Database Connectivity)

> *"JDBC allows Java applications to communicate with relational databases. Developers can execute SQL statements using JDBC connections and prepared statements."* — **Java Database Connectivity (jdbc-01)**

Why it matters for Spring Boot: Spring Boot builds on top of JDBC (and abstracts it further with Spring Data JPA). Knowing JDBC helps you understand **how Spring Boot connects to your database**, executes queries, and manages connections — especially when debugging or configuring a data source.

---

4. 🏗️ MVC Architecture

> *"Model View Controller separates applications into three responsibilities. The Model manages data and business logic, the View handles presentation and the Controller processes requests."* — **MVC Architecture (mvc-01)**

Why it matters for Spring Boot: Spring Boot is built around the **MVC pattern**. Your controllers (`@Controller` / `@RestController`) handle requests, your models represent data, and your services contain business logic. Understanding MVC will make Spring Boot's structure immediately intuitive.

🗺️ Recommended Learning Order

Here's the sequence that makes the most sense given what you already know:

```
Java ✅ → OOP → JDBC → REST API (+ HTTP) → MVC Architecture → Spring Boot 🚀
```

Since you know SQL already, JDBC will feel very natural — it's just the Java bridge to execute the SQL you already know!

📖 Sources Used

Source Type	Details
📊 **Knowledge Graph**	`project-curated-learning-path` — Spring Boot prerequisites and learning path

📄 **Course Documents**	Spring Boot Fundamentals (`spring-01`), REST API Fundamentals (`rest-01`), Understanding HTTP (`http-01`), Java Database Connectivity (`jdbc-01`), OOP Guide (`oop-01`), MVC Architecture (`mvc-01`)

You're well on your way — with Java and SQL already under your belt, you only need to fill in **OOP, REST API, JDBC, and MVC** before diving into Spring Boot. Would you like me to go deeper on any of these topics? 😊